

Pré-Formulário

Programação Web

Do zero ao primeiro CRUD

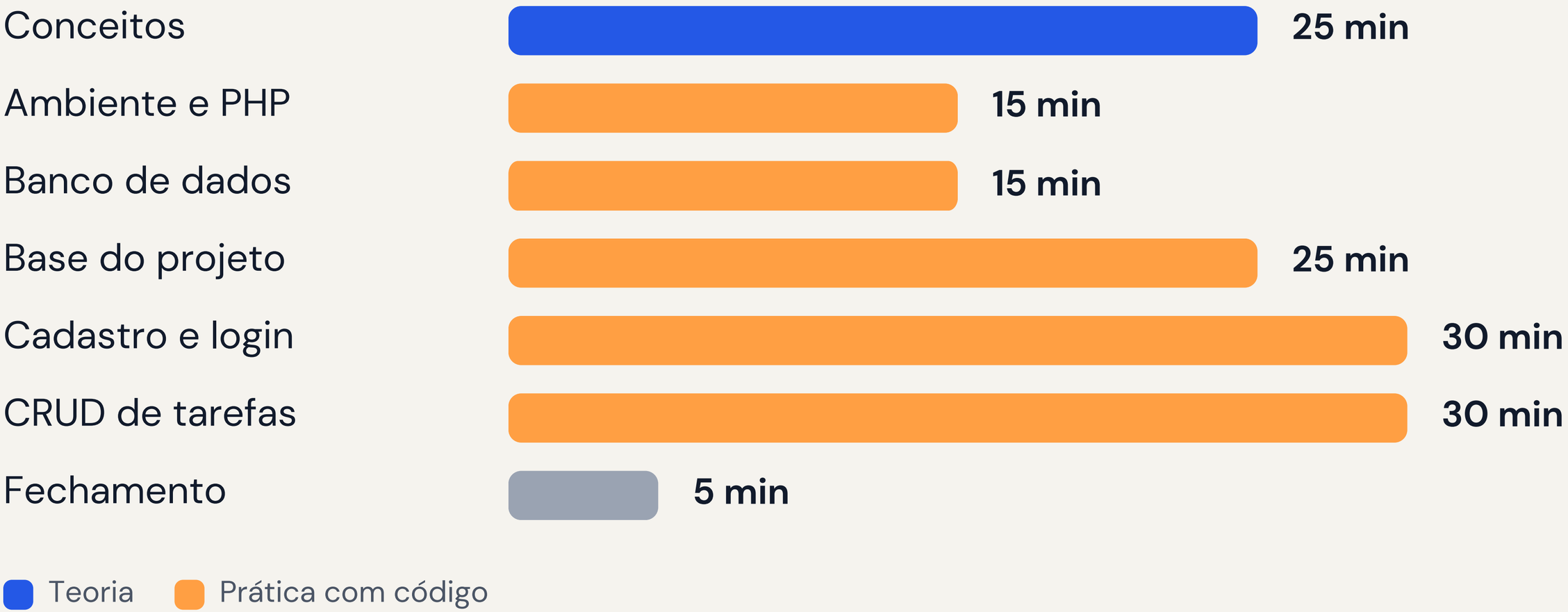
HTML PHP MySQL



Lorenzo Jordani Bertozzi Luz

VISÃO GERAL

Roteiro de 2h30



VISÃO GERAL

O que vamos construir





Cadastro · Login · Sessão · Logout · Criar tarefa · Listar tarefas · Editar tarefa · Excluir tarefa

VISÃO GERAL

CRUD: as quatro operações

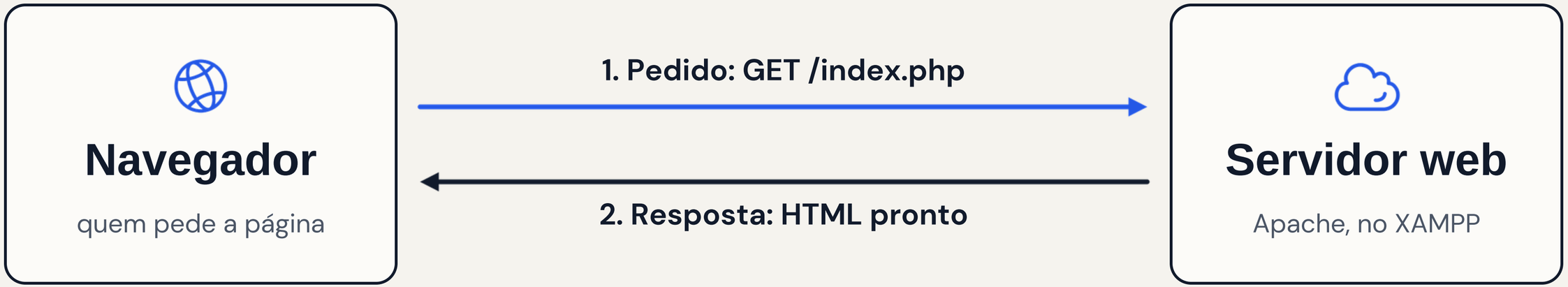


Create	Read	Update	Delete
`INSERT`	`SELECT`	`UPDATE`	`DELETE`
Criar tarefa	Listar tarefas	Editar tarefa	Excluir tarefa

Quase todo sistema que guarda dados repete essas quatro operações.

COMO A WEB FUNCIONA

Pedido e resposta



3. O navegador lê o HTML, aplica o CSS e desenha a página

COMO A WEB FUNCIONA

HTML, CSS e JavaScript

HTML

Estrutura e conteúdo

```
<h1>Olá</h1>
```

Olá

CSS

Aparência

```
h1 { color: blue; }
```

Olá

JavaScript

Comportamento

```
alert('Oi!');
```

Oi! OK

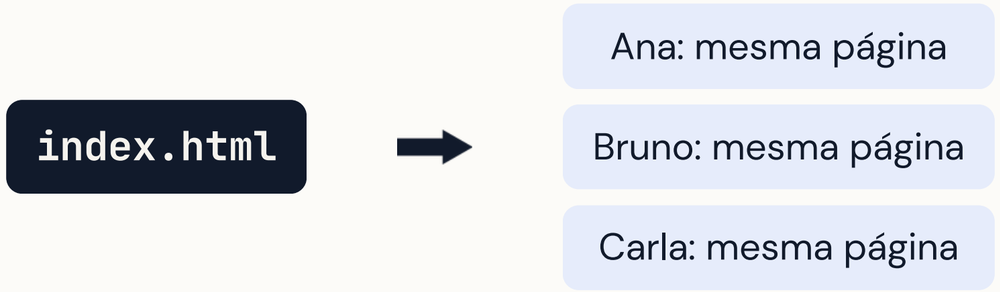
Neste curso o HTML e o CSS chegam prontos para copiar, e o PHP trabalha no servidor.

COMO A WEB FUNCIONA

Página estática e dinâmica

Estática

HTML puro: o mesmo arquivo para todos



Para mudar o conteúdo, alguém edita o arquivo.

Dinâmica

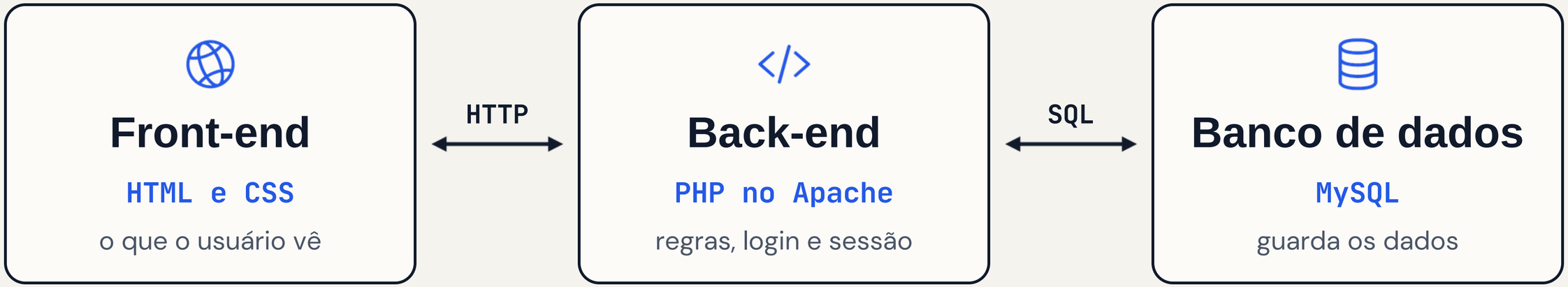
PHP gera o HTML na hora, com dados do banco



O conteúdo muda conforme os dados e o usuário.

COMO A WEB FUNCIONA

Front-end, back-end e banco de dados



Exemplo: o login atravessa as três camadas

1. Formulário de login (HTML) → 2. PHP recebe e valida os dados → 3. MySQL devolve o usuário → 4. PHP responde com a página certa

COMO A WEB FUNCIONA

GET e POST

GET

Pedir uma página ou dados

Os parâmetros aparecem na URL

```
GET /editar.php?id=3
```

Usamos para: abrir a lista e o formulário de edição

POST

Enviar dados de um formulário

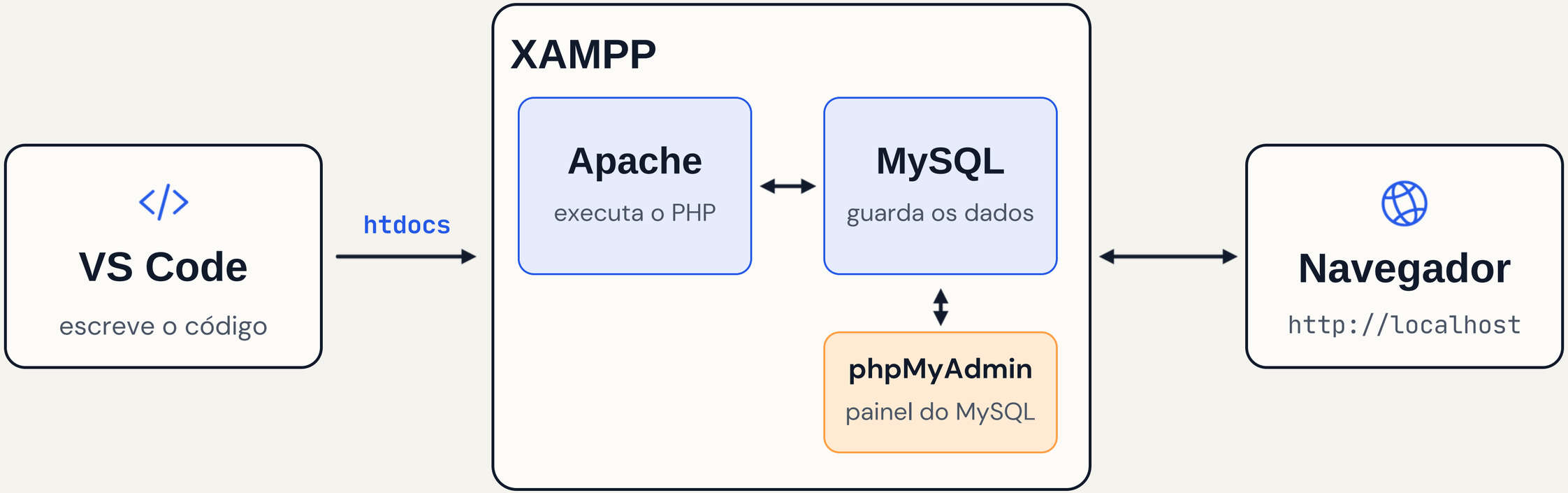
Os dados vão no corpo da requisição

```
POST /login.php
corpo: email=...&senha=...
```

Usamos para: login, cadastro, salvar e excluir

COMO A WEB FUNCIONA

Nossas ferramentas



AMBIENTE

Agora é prática

1. Crie o arquivo com o nome em destaque
2. Copie o código do guia
3. Salve, abra no navegador e teste

login.php 1 / 3

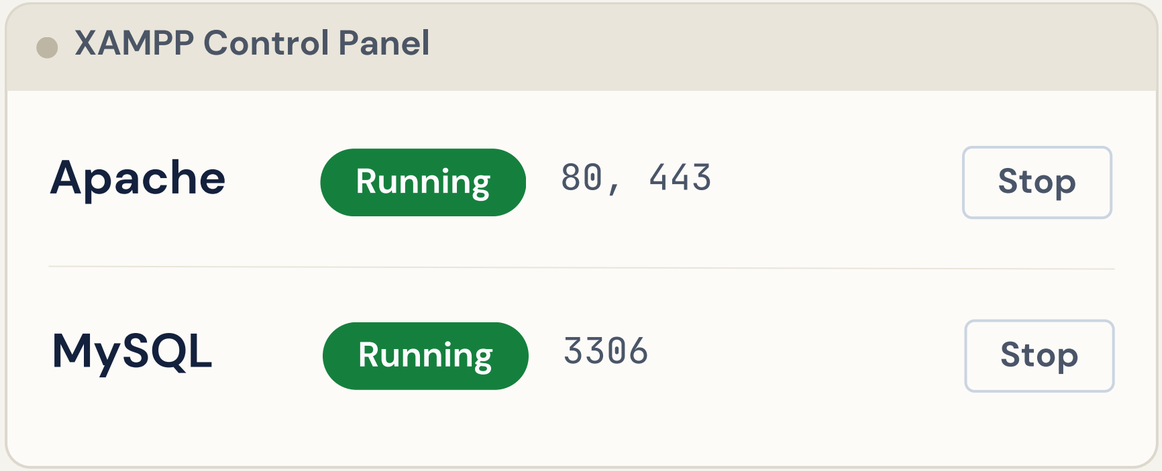
```
<?php
require_once 'funcoes.php';
if (esta_logado()) {
   header('Location: index.php');
   exit;
}
```



Git Hub

AMBIENTE

1. Iniciar o XAMPP



1. Clique em Start no Apache e no MySQL
2. Os nomes ficam com fundo verde
3. Abra no navegador:

```
http://localhost/
```

```
http://localhost/phpmyadmin/
```

⚠ Apache não sobe? Outro programa pode estar usando a porta 80 (Skype, IIS). Feche-o e tente de novo.

AMBIENTE

2. Criar a pasta do projeto

xampp

htdocs

crud_tarefas

`teste.php`

ONDE FICA A HTDOCS

Windows	`C:\xampp\htdocs`
Linux	`/opt/lampp/htdocs`
macOS	`/Applications/XAMPP/htdocs`

No VS Code: **File › Open Folder** e escolha a pasta **crud_tarefas**

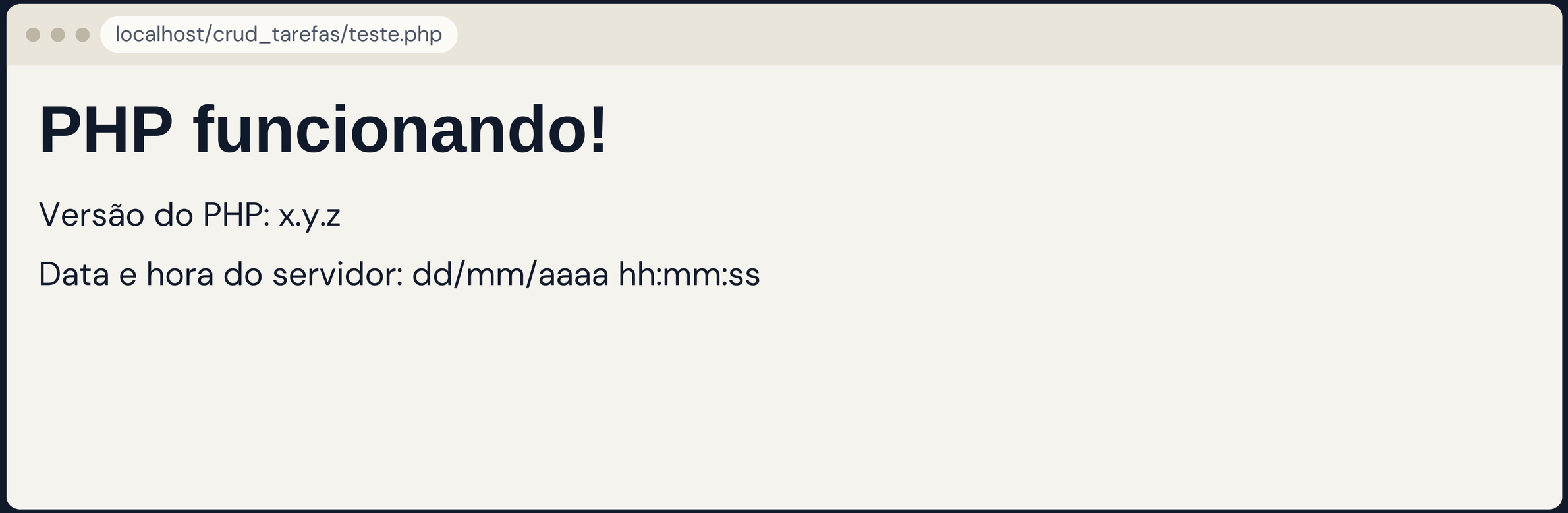
O endereço do projeto será

http://localhost/crud_tarefas/

teste.php 3. Testar o PHP http://localhost/crud_tarefas/teste.php

```php
<?php
echo "<h1>PHP funcionando!</h1>";
echo "<p>Versão do PHP: " . phpversion() . "</p>";
echo "<p>Data e hora do servidor: " . date('d/m/Y H:i:s') . "</p>";
?>
```

RESULTADO ESPERADO NO NAVEGADOR



AMBIENTE

Se algo não funcionou

O Apache não inicia

CAUSA

Outro programa usa a porta 80 (Skype, IIS)

SOLUÇÃO

Feche o programa ou troque a porta em Config > Apache (httpd.conf)

Aparece o código PHP na tela

CAUSA

O arquivo foi aberto direto do disco (file:///...)

SOLUÇÃO

Use sempre o endereço http://localhost/...

Erro 404 (Not Found)

CAUSA

Pasta ou nome do arquivo errado

SOLUÇÃO

Confira se o arquivo está em htdocs/crud_tarefas

03

Banco de dados MySQL

Tabelas, INSERT e SELECT no phpMyAdmin

15 min

BANCO DE DADOS

phpMyAdmin



1. Abra o phpMyAdmin
2. Clique na aba SQL
3. Cole o script
4. Clique em Executar

BANCO DE DADOS

Modelo do banco

usuarios	
id PK	INT
nome	VARCHAR(100)
email	VARCHAR(150) · único
senha	VARCHAR(255) · hash

usuario_id aponta para o id
do dono da tarefa

1 usuário



N tarefas

tarefas	
id PK	INT
usuario_id FK	INT
titulo	VARCHAR(150)
descricao	TEXT
status	ENUM · 3 valores
criado_em	TIMESTAMP
atualizado_em	TIMESTAMP

Apagar um usuário apaga também as tarefas dele (ON DELETE CASCADE).

banco.sql Tabela de tarefas

```sql
CREATE DATABASE IF NOT EXISTS crud_tarefas
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE crud_tarefas;

CREATE TABLE IF NOT EXISTS usuarios (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    senha VARCHAR(255) NOT NULL
);
```

utf8mb4
Aceita acentos e emojis

AUTO_INCREMENT
O id é gerado sozinho

UNIQUE
Sem e-mail repetido

senha
Guarda o hash, nunca a senha digitada

banco.sql Tabela de tarefas

```sql
CREATE TABLE IF NOT EXISTS tarefas (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    usuario_id INT UNSIGNED NOT NULL,
    titulo VARCHAR(150) NOT NULL,
    descricao TEXT NULL,
    status ENUM('pendente', 'andamento', 'concluida') NOT NULL DEFAULT 'pendente',
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_tarefas_usuario
        FOREIGN KEY (usuario_id) REFERENCES usuarios(id)
        ON DELETE CASCADE
);
```

ENUM
Limita o status a três valores

FOREIGN KEY
Liga cada tarefa ao seu usuário

CASCADE
Apagar o usuário apaga as tarefas dele

ON UPDATE
atualizado_em muda sozinho

phpMyAdmin › aba SQL

```
USE crud_tarefas;

INSERT INTO usuarios (nome, email, senha)
VALUES ('Usuário de Teste', 'teste@exemplo.com', 'senha');
INSERT INTO tarefas (usuario_id, titulo, descricao, status) VALUES
(1, 'Instalar o XAMPP', 'Baixar e iniciar Apache e MySQL', 'concluida'),
(1, 'Criar o banco de dados', 'Rodar o script SQL no phpMyAdmin', 'andamento'),
(1, 'Fazer o primeiro CRUD', 'Listar, criar, editar e excluir tarefas', 'pendente');
```

Resultado na tabela tarefas

id	usuario_id	titulo	status
1	1	Instalar o XAMPP	concluida
2	1	Criar o banco de dados	andamento
3	1	Fazer o primeiro CRUD	pendente

SELECT: consultando o banco

phpMyAdmin › aba SQL

```sql
-- Todas as colunas e todas as linhas
SELECT * FROM usuarios;
SELECT * FROM tarefas;
-- Só algumas colunas, com filtro
SELECT id, titulo, status FROM tarefas WHERE status = 'pendente';
-- Ordenando: mais recentes primeiro
SELECT * FROM tarefas WHERE usuario_id = 1 ORDER BY criado_em DESC;
-- Juntando as duas tabelas
SELECT t.titulo, t.status, u.nome
FROM tarefas t
INNER JOIN usuarios u ON u.id = t.usuario_id;
```

SELECT
quais colunas

FROM
de qual tabela

WHERE
quais linhas

ORDER BY
em que ordem

UPDATE, DELETE e limpeza

phpMyAdmin › aba SQL · laranja = ponto-chave

```
UPDATE tarefas SET status = 'concluida' WHERE id = 2;
DELETE FROM tarefas WHERE id = 3;
-- Limpar os dados de teste (antes de seguir)
DELETE FROM tarefas;
DELETE FROM usuarios;
ALTER TABLE tarefas AUTO_INCREMENT = 1;
ALTER TABLE usuarios AUTO_INCREMENT = 1;
```

⚠ **Sempre use WHERE em UPDATE e DELETE. Sem ele, o comando atinge todas as linhas.**

As quatro últimas linhas apagam os dados de teste: o usuário de exemplo tem uma senha falsa e não consegue entrar no sistema.

04

Arquivos e conexão

Organização do projeto e PHP falando com o MySQL

10 min

Mapa dos arquivos do projeto

Banco e funções	conexao.php · funcoes.php Acesso ao MySQL, sessão, validação e funções auxiliares.
Estrutura visual	topo.php · rodape.php · style.css Partes reutilizadas em todas as páginas.
Autenticação	cadastro.php · login.php · logout.php Criar conta, entrar e sair.
Tarefas (CRUD)	index.php · form_tarefa.php · criar.php · editar.php · excluir.php Listar, criar, alterar e remover tarefas.

Crie cada arquivo exatamente com o nome indicado. Os arquivos PHP ficam todos dentro de crud_tarefas.

Conectando ao MySQL

conexao.php

```php
<?php
$host  = 'localhost';
$user  = 'root';
$pass  = ''; // padrão do XAMPP local
$banco = 'crud_tarefas';

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

try {
    $conn = new mysqli($host, $user, $pass, $banco);
    $conn->set_charset('utf8mb4');
} catch (mysqli_sql_exception $e) {
    // Em produção, registre o erro no log.
    die('Erro ao conectar ao banco de dados.');
}
```



Se o MySQL local tiver senha, ajuste $pass. Não exponha detalhes de erro em sistemas publicados.

Testando a conexão

teste.php

```php
<?php
require_once 'conexao.php';

$resultado = $conn->query(
    'SELECT id, titulo, status FROM tarefas'
);

echo '<h1>Tarefas no banco</h1><ul>';

while ($linha = $resultado->fetch_assoc()) {
    echo '<li>' . htmlspecialchars($linha['titulo'])
        . ' (' . htmlspecialchars($linha['status']) . ')</li>';
}

echo '</ul>';
```

⚠ **Abra http://localhost/crud_tarefas/teste.php. Lista vazia também significa que a conexão funcionou.**

Três cuidados ao acessar o banco

require_once	Inclui um arquivo uma única vez. Evita repetir conexão ou funções.
Prepared statements	Use ? e bind_param() para separar os dados do usuário do comando SQL.
Escapar a saída	Use htmlspecialchars() antes de exibir texto digitado pelo usuário no HTML.

Essas práticas ajudam a evitar SQL Injection e XSS. Não concatene dados de formulário diretamente no SQL.

Consulta preparada: exemplo

trecho reutilizável em PHP

```
$stmt = $conn->prepare(
    'SELECT id, titulo FROM tarefas
     WHERE usuario_id = ? AND status = ?'
);

$stmt->bind_param('is', $usuarioId, $status);
$stmt->execute();

$resultado = $stmt->get_result();
$linha = $resultado->fetch_assoc();
```

⚠ **Abra http://localhost/crud_tarefas/teste.php. Lista vazia também significa que a conexão funcionou.**

05

Arquivos de base

Sessão, cabeçalho, rodapé e aparência

15 min

Funções compartilhadas · parte 1/3

funcoes.php

```php
<?php
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

const STATUS_TAREFA = [
    'pendente'  => 'Pendente',
    'andamento' => 'Em andamento',
    'concluida' => 'Concluída',
];

function esta_logado(): bool
{
    return isset($_SESSION['usuario_id']);
}
```

⚠ **Crie funcoes.php. A função exigir_login() protege as páginas privadas; e() escapa textos antes de exibi-los.**

Funções compartilhadas · parte 2/3

funcoes.php

```php
function exigir_login(): void
{
    if (!esta_logado()) {
        header('Location: login.php');
        exit;
    }
}

function e(?string $texto): string
{
    return htmlspecialchars($texto ?? '', ENT_QUOTES, 'UTF-8');
}
```

Funções compartilhadas · parte 3/3

funcoes.php

```
function validar_tarefa(array $dados): array
{
    $erros = [];

    if (empty($dados['titulo'])) {
        $erros[] = 'Informe o título da tarefa.';
    }

    elseif (mb_strlen($dados['titulo']) > 150) {
        $erros[] = 'O título pode ter no máximo 150 caracteres.';
    }

    if (!array_key_exists($dados['status'] ?? '', STATUS_TAREFA)) {
        $erros[] = 'Status inválido.';
    }

    return $erros;
}
```

Cabeçalho reutilizável · parte 1/2

topo.php

```php
<?php
require_once __DIR__ . '/funcoes.php';
$tituloPagina = $tituloPagina ?? 'Tarefas';
?>
<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title><?= e($tituloPagina) ?> | Minhas Tarefas</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
```

⚠ **Defina $tituloPagina antes de incluir topo.php. Não imprima nada antes de header() ou redirecionamentos.**

Cabeçalho reutilizável · parte 2/3

topo.php

```
<header class="topo">
  <div class="container topo-conteudo">
    <a class="marca" href="index.php">Minhas Tarefas</a>
    <nav>
      <?php if (esta_logado()): ?>
        <span class="saudacao">Olá, <?= e($_SESSION['usuario_nome']) ?></span>
        <a href="index.php">Tarefas</a>
        <a href="logout.php">Sair</a>
      <?php else: ?>
        <a href="login.php">Entrar</a>
        <a href="cadastro.php">Cadastrar</a>
      <?php endif; ?>
    </nav>
  </div>
</header>
<main class="container">
```

Rodapé reutilizável

rodape.php

```
</main>
<footer class="rodape">
  <div class="container">Meu primeiro CRUD com PHP e MySQL</div>
</footer>
</body>
</html>
```

⚠ **O rodape.php fecha as tags abertas em topo.php. As páginas só precisam incluir os dois arquivos.**

Estilo compartilhado

`style.css`

GIT HUB

Copie o codigo no repositório

URL//:

```
https://github.com/LorenzoBertozzi/Meu-primeiro-CRUD/blob/main/crud_tarefas/style.css
```

Usamos para: configurar todo estilização de nosso site

⚠ **Cole este primeiro bloco no style.css. O próximo slide continua no mesmo arquivo.**

06

Cadastro e login

Criar conta, proteger senhas e manter a sessão

30 min

Cadastro de usuário · parte 1/7

cadastro.php

```
<?php
require_once 'funcoes.php';
require_once 'conexao.php';

if (esta_logado()) {
    header('Location: index.php');
    exit;
}

$erros = [];
$nome = '';
$email = '';
```

⚠ Crie cadastro.php. Este é o bloco inicial; o próximo slide continua após o fechamento do PHP.

Cadastro de usuário · parte 2/7

cadastro.php

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $nome = trim($_POST['nome'] ?? '');
    $email = trim($_POST['email'] ?? '');
    $senha = $_POST['senha'] ?? '';
    $senha2 = $_POST['senha2'] ?? '';

    if ($nome === '') $erros[] = 'Informe seu nome.';
    if (!filter_var($email, FILTER_VALIDATE_EMAIL))
        $erros[] = 'Informe um e-mail válido.';
    if (strlen($senha) < 6)
        $erros[] = 'A senha deve ter pelo menos 6 caracteres.';
    if ($senha !== $senha2)
        $erros[] = 'As senhas não conferem.';
```

⚠ **Crie cadastro.php. Este é o bloco inicial; o próximo slide continua após o fechamento do PHP.**

Cadastro de usuário · parte 3/7

cadastro.php

```
if (empty($erros)) {
    $stmt = $conn->prepare('SELECT id FROM usuarios WHERE email = ?');
    $stmt->bind_param('s', $email);
    $stmt->execute();
    $stmt->store_result();
    if ($stmt->num_rows > 0)
        $erros[] = 'Este e-mail já está cadastrado.';
    $stmt->close();
}
```

⚠ **Crie cadastro.php. Este é o bloco inicial; o próximo slide continua após o fechamento do PHP.**

Cadastro de usuário · parte 4/7

cadastro.php

```
    if (empty($erros)) {
        $hash = password_hash($senha, PASSWORD_DEFAULT);
        $stmt = $conn->prepare(
            'INSERT INTO usuarios (nome, email, senha) VALUES (?, ?, ?)'
        );
        $stmt->bind_param('sss', $nome, $email, $hash);
        $stmt->execute();
        $stmt->close();
        header('Location: login.php?cadastro=ok');
        exit;
    }
}
$tituloPagina = 'Cadastro';
require 'topo.php';
?>
```

⚠ **Crie cadastro.php. Este é o bloco inicial; o próximo slide continua após o fechamento do PHP.**

Cadastro de usuário · parte 5/7

cadastro.php

```
<section class=" card card-auth">
 <h1>Criar conta</h1>

 <?php if (!empty($erros)): ?>
    <div class="alerta alerta-erro"><ul>
    <?php foreach ($erros as $erro): ?>
    <li><?= e($erro) ?></li>
 <?php endforeach; ?>
    </ul></div>
 <?php endif; ?>
```

⚠ Os dois blocos formam um único arquivo cadastro.php. password_hash() guarda um hash, nunca a senha em texto puro.

Cadastro de usuário · parte 6/7

cadastro.php

```
<form method="post" action="cadastro.php">
    <div class="campo">
        <label for="nome">Nome</label>
            <input type="text" id="nome" name="nome" value="<?= e($nome) ?>" required>
    </div>
    <div class="campo">
        <label for="email">E-mail</label>
            <input type="email" id="email" name="email" value="<?= e($email) ?>" required>
    </div>
    <div class="campo">
        <label for="senha">Senha</label>
            <input type="password" id="senha" name="senha" minlength="6" required>
    </div>
```

⚠ Os dois blocos formam um único arquivo cadastro.php. password_hash() guarda um hash, nunca a senha em texto puro.

Cadastro de usuário · parte 7/7

cadastro.php

```
        <div class="campo">
            <label for="senha2">Repita a senha</label>
            <input type="password" id="senha2" name="senha2" minlength="6" required>
        </div>
        <button type="submit" class="btn btn-primario btn-bloco">Cadastrar</button>
    </form>
    <p class="texto-form">Já tem conta? <a href="login.php">Entrar</a></p>
    </section>
    <?php require 'rodape.php'; ?>
```

⚠ **Os dois blocos formam um único arquivo cadastro.php. password_hash() guarda um hash, nunca a senha em texto puro.**

Login · parte 1/5

login.php

```php
<?php
require_once 'funcoes.php';
require_once 'conexao.php';

if (esta_logado()) {
    header('Location: index.php');
    exit;
}

$erro = '';
$email = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $email = trim($_POST['email'] ?? '');
    $senha = $_POST['senha'] ?? '';
```

⚠ **password_verify() compara a senha digitada com o hash. Não tente descriptografar o hash.**

Login · parte 2/5

login.php

```
$stmt = $conn->prepare(
    'SELECT id, nome, senha FROM usuarios WHERE email = ?'
);
$stmt->bind_param('s', $email);
$stmt->execute();
$usuario = $stmt->get_result()->fetch_assoc();
$stmt->close();

if ($usuario && password_verify($senha, $usuario['senha'])) {
    session_regenerate_id(true);
    $_SESSION['usuario_id'] = $usuario['id'];
    $_SESSION['usuario_nome'] = $usuario['nome'];
    header('Location: index.php');
    exit;
}
```

⚠ **password_verify() compara a senha digitada com o hash. Não tente descriptografar o hash.**

Login · parte 3/5

login.php

```
    $erro = 'E-mail ou senha incorretos.';
}
$tituloPagina = 'Entrar';
require 'topo.php';
?>
```

⚠ **password_verify() compara a senha digitada com o hash. Não tente descriptografar o hash.**

Login · parte 4/5

login.php

```
<section class=" card card-auth">
 <h1>Entrar</h1>

 <?php if (($_GET['cadastro'] ?? '') === 'ok'): ?>
 <div class="alerta alerta-sucesso">
 Conta criada! Entre com seu e-mail e senha.
 </div>
 <?php endif; ?>

 <?php if ($erro !== ''): ?>
 <div class="alerta alerta-erro"><?= e($erro) ?></div>
 <?php endif; ?>
```

⚠ **A mensagem de erro é genérica para não revelar se determinado e-mail existe.**

Login · parte 5/5

login.php

```
 <form method="post" action="login.php">
    <div class="campo">
       <label for="email">E-mail</label>
       <input type="email" id="email" name="email" value="<?= e($email) ?>" required>
    </div>
    <div class="campo">
       <label for="senha">Senha</label>
       <input type="password" id="senha" name="senha" required>
    </div>
    <button type="submit" class="btn btn-primario btn-bloco">Entrar</button>
 </form>
 <p class="texto-form">Ainda não tem conta? <a href="cadastro.php">Cadastre-se</a></p>
</section>
<?php require 'rodape.php'; ?>
```

⚠ **A mensagem de erro é genérica para não revelar se determinado e-mail existe.**

Logout: encerrar a sessão

logout.php

```php
<?php
require_once 'funcoes.php';

// Limpa os dados da sessão
$_SESSION = [];

// Remove o cookie da sessão no navegador
if (ini_get('session.use_cookies')) {
    $p = session_get_cookie_params();
    setcookie(
        session_name(), '', time() - 42000,
        $p['path'], $p['domain'], $p['secure'], $p['httponly']
    );
}

// Destrói a sessão no servidor
session_destroy();

header('Location: login.php');
exit;
```

07

CRUD de tarefas

Listar, criar, editar e excluir dados

30 min

Listar tarefas · parte 1/4

index.php

```php
<?php
require_once 'funcoes.php';
require_once 'conexao.php';
exigir_login();

$usuarioId = $_SESSION['usuario_id'];

$stmt = $conn->prepare(
    'SELECT id, titulo, descricao, status, criado_em
     FROM tarefas
     WHERE usuario_id = ?
     ORDER BY criado_em DESC'
);
$stmt->bind_param('i', $usuarioId);
$stmt->execute();
$tarefas = $stmt->get_result();
```

⚠ **Crie index.php. O filtro WHERE usuario_id = ? É essencial: cada usuário só vê suas tarefas.**

Listar tarefas · parte 2/4

index.php

```
$mensagens = [
    'criada'   => 'Tarefa criada com sucesso.',
    'editada'  => 'Tarefa atualizada com sucesso.',
    'excluida' => 'Tarefa excluída com sucesso.',
];
$msg = $mensagens[$_GET['msg'] ?? ''] ?? '';

$tituloPagina = 'Minhas tarefas';
require 'topo.php';
?>
<div class="cabecalho-pagina">
  <h1>Minhas tarefas</h1>
  <a class="btn btn-primario" href="criar.php">+ Nova tarefa</a>
</div>
<?php if ($msg !== ''): ?>
  <div class="alerta alerta-sucesso"><?= e($msg) ?></div>
<?php endif; ?>
```

⚠ **Crie index.php. O filtro WHERE usuario_id = ? É essencial: cada usuário só vê suas tarefas.**

Listar tarefas · parte 3/4

index.php

```
<?php if ($tarefas->num_rows === 0): ?>
 <div class=" card vazio">
     Você ainda não tem tarefas. Clique em Nova tarefa para começar.
 </div>
<?php else: ?>
 <div class="card"><table class="tabela">
     <thead><tr>
     <th>Tarefa</th><th>Status</th><th>Criada em</th><th>Ações</th>
     </tr></thead>
     <tbody>
     <?php while ($t = $tarefas->fetch_assoc()): ?>
     <tr>
          <td>
               <strong><?= e($t['titulo']) ?></strong>
               <?php if (!empty($t['descricao'])): ?>
                    <div class="descricao"><?= e($t['descricao']) ?></div>
               <?php endif; ?>
          </td>
```

⚠ Este é o restante do mesmo index.php. While percorre as linhas retornadas pelo banco.

Listar tarefas · parte 4/4

index.php

```
 <td><span class="badge badge-<?= e($t['status']) ?>">
     <?= e(STATUS_TAREFA[$t['status']]) ?>
     </span></td>
     <td><?= date('d/m/Y H:i', strtotime($t['criado_em'])) ?></td>
        <td class="acoes">
           <a class="btn btn-secundario btn-pequeno" href="editar.php?id=<?= (int)$t['id']
?>">Editar</a>
           <form method="post" action="excluir.php" onsubmit="return confirm('Excluir esta
tarefa?');">
              <input type="hidden" name="id" value="<?= (int)$t['id'] ?>">
              <button type="submit" class="btn btn-perigo btn-pequeno">Excluir</button>
           </form>
        </td>
 </tr>
 <?php endwhile; ?>
 </tbody>
 </table></div>
<?php endif; ?>
<?php require 'rodape.php'; ?>
```

Formulário compartilhado · parte 1/2

form_tarefa.php

```
<?php if (!empty($erros)): ?>
    <div class="alerta alerta-erro"><ul>
    <?php foreach ($erros as $erro): ?>
    <li><?= e($erro) ?></li>
    <?php endforeach; ?>
    </ul></div>
<?php endif; ?>

<form method="post">
    <div class="campo">
       <label for="titulo">Título</label>
       <input type="text" id="titulo" name="titulo" maxlength="150"
       value="<?= e($tarefa['titulo']) ?>" required>
    </div>
    <div class="campo">
       <label for="descricao">Descrição</label>
       <textarea id="descricao" name="descricao" rows="4"><?= e($tarefa['descricao']) ?>
       </textarea>
```

⚠ **Criar e editar incluem este arquivo, evitando duplicar os campos do formulário.**

Formulário compartilhado · parte 2/2

form_tarefa.php

```
  </div>
  <div class="campo">
      <label for="status">Status</label>
      <select id="status" name="status">
          <?php foreach (STATUS_TAREFA as $valor => $rotulo): ?>
          <option value="<?= e($valor) ?>"
              <?= $tarefa['status'] === $valor ? 'selected' : '' ?>>
              <?= e($rotulo) ?>
          </option>
          <?php endforeach; ?>
      </select>
  </div>
  <div class="form-acoes">
      <button type="submit" class="btn btn-primario"><?= e($textoBotao) ?></button>
      <a class="btn btn-secundario" href="index.php">Cancelar</a>
  </div>
</form>
```

⚠ **Criar e editar incluem este arquivo, evitando duplicar os campos do formulário.**

Formulário compartilhado · parte 2/2

form_tarefa.php

```
  </div>
  <div class="campo">
      <label for="status">Status</label>
      <select id="status" name="status">
          <?php foreach (STATUS_TAREFA as $valor => $rotulo): ?>
          <option value="<?= e($valor) ?>"
              <?= $tarefa['status'] === $valor ? 'selected' : '' ?>>
              <?= e($rotulo) ?>
          </option>
          <?php endforeach; ?>
      </select>
  </div>
  <div class="form-acoes">
      <button type="submit" class="btn btn-primario"><?= e($textoBotao) ?></button>
      <a class="btn btn-secundario" href="index.php">Cancelar</a>
  </div>
</form>
```

⚠ **Criar e editar incluem este arquivo, evitando duplicar os campos do formulário.**

Criar tarefa (Create) · parte 1/3

criar.php

```php
<?php
require_once 'funcoes.php';
require_once 'conexao.php';
exigir_login();

$erros = [];
$tarefa = ['titulo' => '', 'descricao' => '', 'status' => 'pendente'];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $tarefa['titulo'] = trim($_POST['titulo'] ?? '');
    $tarefa['descricao'] = trim($_POST['descricao'] ?? '');
    $tarefa['status'] = $_POST['status'] ?? 'pendente';
    $erros = validar_tarefa($tarefa);
```

⚠ O dono da tarefa vem da sessão, nunca de um campo que o navegador possa alterar.

Criar tarefa (Create) · parte 2/3

criar.php

```
    if (empty($erros)) {
        $usuarioId = $_SESSION['usuario_id'];
        $stmt = $conn->prepare(
            'INSERT INTO tarefas (usuario_id, titulo, descricao, status)
             VALUES (?, ?, ?, ?)'
        );
        $stmt->bind_param(
            'isss', $usuarioId, $tarefa['titulo'],
            $tarefa['descricao'], $tarefa['status']
        );
        $stmt->execute();
        $stmt->close();
        header('Location: index.php?msg=criada');
        exit;
    }
}
```

⚠ O dono da tarefa vem da sessão, nunca de um campo que o navegador possa alterar.

Criar tarefa (Create) · parte 3/3

criar.php

```
$tituloPagina = 'Nova tarefa';
$textoBotao = 'Salvar tarefa';
require 'topo.php';
?>
<section class="card">
  <h1>Nova tarefa</h1>
  <?php require 'form_tarefa.php'; ?>
</section>
<?php require 'rodape.php'; ?>
```



O dono da tarefa vem da sessão, nunca de um campo que o navegador possa alterar.

Editar tarefa (Update) · parte 1/4

editar.php

```php
<?php
require_once 'funcoes.php';
require_once 'conexao.php';
exigir_login();

$id = (int)($_GET['id'] ?? 0);
$usuarioId = $_SESSION['usuario_id'];
$erros = [];

$stmt = $conn->prepare(
    'SELECT id, titulo, descricao, status
     FROM tarefas WHERE id = ? AND usuario_id = ?'
);
$stmt->bind_param('ii', $id, $usuarioId);
$stmt->execute();
$tarefa = $stmt->get_result()->fetch_assoc();
$stmt->close();

if (!$tarefa) {
```

⚠ O filtro usuario_id protege contra alguém alterar o id na URL para tentar editar a tarefa de outra pessoa.

Editar tarefa (Update) · parte 2/4

editar.php

```
if (!$tarefa) {
    header('Location: index.php');
    exit;
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $tarefa['titulo'] = trim($_POST['titulo'] ?? '');
    $tarefa['descricao'] = trim($_POST['descricao'] ?? '');
    $tarefa['status'] = $_POST['status'] ?? 'pendente';
    $erros = validar_tarefa($tarefa);
```



O filtro usuario_id protege contra alguém alterar o id na URL para tentar editar a tarefa de outra pessoa.

Editar tarefa (Update) · parte 3/4

editar.php

```
    if (empty($erros)) {
        $stmt = $conn->prepare(
            'UPDATE tarefas
             SET titulo = ?, descricao = ?, status = ?
             WHERE id = ? AND usuario_id = ?'
        );
        $stmt->bind_param(
            'sssii', $tarefa['titulo'], $tarefa['descricao'],
            $tarefa['status'], $id, $usuarioId
        );
        $stmt->execute();
        $stmt->close();
        header('Location: index.php?msg=editada');
        exit;
    }
}
```



O filtro usuario_id protege contra alguém alterar o id na URL para tentar editar a tarefa de outra pessoa.

Editar tarefa (Update) · parte 4/4

editar.php

```
$tituloPagina = 'Editar tarefa';
$textoBotao = 'Salvar alterações';
require 'topo.php';
?>
<section class="card">
  <h1>Editar tarefa</h1>
  <?php require 'form_tarefa.php'; ?>
</section>
<?php require 'rodape.php'; ?>
```



Este trecho continua no mesmo arquivo editar.php, logo após o bloco PHP do slide anterior.

Excluir tarefa (Delete)

excluir.php

```php
<?php
require_once 'funcoes.php';
require_once 'conexao.php';
exigir_login();

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    header('Location: index.php');
    exit;
}

$id = (int)($_POST['id'] ?? 0);
$usuarioId = $_SESSION['usuario_id'];

$stmt = $conn->prepare('DELETE FROM tarefas WHERE id =? AND usuario_id = ?');
$stmt->bind_param('ii', $id, $usuarioId);
$stmt->execute();
$stmt->close();

header('Location: index.php?msg=excluida');
exit;
```

08

Testes e próximos passos

Verifique o funcionamento e resolva erros comuns

10 min

Checklist: teste o sistema

1 · Conta	Cadastre uma conta. Teste e-mail repetido, senhas diferentes e senha curta.
2 · Login	Tente uma senha errada e depois a correta. Confirme que abre a lista.
3 · CRUD	Crie uma tarefa, altere o status, edite o título e exclua.
4 · Segurança	Saia e tente abrir index.php. Crie outra conta e confirme que não vê as tarefas anteriores.

Endereço inicial: http://localhost/crud_tarefas/cadastro.php

Erros comuns: onde olhar?

Access denied	Usuário ou senha do MySQL diferente do configurado em conexao.php.
Unknown database	Execute o script SQL e confira se o nome é crud_tarefas.
Página em branco	Confira os logs do Apache no XAMPP e procure erros de PHP.
Acentos quebrados	Salve os arquivos em UTF-8; confira charset no HTML e no MySQL.

Se header() reclamar que os cabeçalhos já foram enviados, verifique se houve saída antes do redirecionamento.

O que você construiu?

HTML + CSS	Formulários, tabela e interface.
PHP	Validação, autenticação, sessão e regras do sistema.
SQL + MySQL	Estrutura, consulta, inserção, alteração e exclusão de dados.
Próximo passo	Estude validação, segurança, permissões, organização do código e publicação.

O exemplo é para estudo local. Antes de publicar, configure credenciais, erros, HTTPS e proteções adicionais.



Pós-Formulário

08

Testes e próximos passos

Verifique o funcionamento e resolva erros comuns

10 min